

Freight Rate Prediction: Validation and Split Report

Melis Kılıç

1. The data

Development data	<code>train_test.csv</code> , 48,000 loads, 1 January to 31 October 2025
Prediction set	<code>validation.csv</code> , 12,000 loads, 1 November to 31 December 2025
Target	<code>posted_rate</code> , in dollars
Columns	<code>lane</code> (<code>pickup</code> , <code>delivery</code> + coordinates), <code>distance</code> , <code>equipment</code> , <code>weight</code> , <code>date</code> , <code>market_index</code> , <code>quote_signal</code>

The two files do not overlap in time. The development data ends on 31 October and the prediction set starts on 1 November. Every choice in this report follows from that: the model has to forecast two months nobody has seen yet.

Distance explains most of the rate (correlation 0.91) and the median rate per mile is \$2.15, inside a narrow band. Equipment is the main modifier, and Reefer is the most expensive.

2. Validating the data

Exploration turned up four problems, and each one is handled so that no prediction is lost. **Cleaning applies to the training data only. No row is dropped from the validation set, so all 12,000 loads get a rate.**

Problem	Evidence	Handling
Negative weight	292 loads carry a negative weight, which is physically impossible, and the true value cannot be recovered	Treated as missing. LightGBM handles missing values natively, and weight barely moves the price
Wrong distance	<code>distance</code> is a median 1.18x the straight-line distance between the two cities, but 111 loads exceed 1.6x	Removed from training only
Extreme rates	671 loads fall outside \$1.00 to \$3.50 per mile, spread evenly across all months with no pattern behind them	Removed from training only
Uninformative columns	<code>market_index</code> has no relation to price, and <code>quote_signal</code> flips direction from month to month. Neither exists in the December chart file	Dropped

Noise removal costs 779 rows, **1.6% of the training data**, leaving 47,221 loads to train on.

Two checks came back clean: every city maps to exactly one coordinate pair, and no coordinate is missing. The coordinates are shifted from the real map, so they are used as relative positions only and never joined against external map data.

3. The split

Train on January to August, score on September and October. A strict time-based split with no overlap, so that the holdout mirrors the real task: fit on the past, predict months the model has not seen.

A random split was rejected. Loads from the same day would land on both sides of the split and the model would be scored on a day it had already partly memorized. That produces an optimistic number which says nothing about November and December.

The final model behind `validation_predictions.csv` is refit on the **full ten months**, January to October, using the configuration the holdout validated. Nothing is held back at that point, since the most recent months are the most informative ones for November and December.

4. Validation results

September-October holdout, 9,523 loads. The baseline takes the median rate per mile of each equipment type and multiplies it by distance.

	MAE	MAPE	RMSE
Baseline (median \$/mile by equipment)	\$228.74	10.49%	\$670.44
LightGBM	\$97.97	4.32%	\$631.03

MAE is the decision metric. These predictions price loads, and an error costs money in both directions: too high loses the load, too low loses the margin. MAE reads directly as dollars per quote and is not dominated by the handful of random extreme loads. RMSE is dominated by them, which is why the two columns disagree so sharply. It is reported, but no decision rests on it.

Where the error sits

Group		MAE	Loads
Equipment	Dry Van	\$96.65	5,360
	Flatbed	\$95.26	1,770
	Reefer	\$102.95	2,393
Distance	0-250 mi	\$19.03	550
	250-500 mi	\$47.61	1,540
	500-1,000 mi	\$65.53	2,972
	1,000-2,000 mi	\$134.16	2,927
	2,000+ mi	\$170.64	1,534
Load type	Clean	\$42.03	9,363
	Noise (would be dropped from training)	\$3,371.75	160

The error is flat across equipment and grows with distance, as expected: a longer load is a bigger number, and \$170 on a 2,000-mile load is a smaller relative error than \$19 on a 200-mile one.

The last row matters most. On the 9,363 clean loads the model sits at **\$42.03**, and the headline \$97.97 is carried by 160 loads (1.7%) that are noise of the same kind removed from training. They are kept in the holdout on purpose, because the real validation set cannot be filtered either.

What the model leans on

Share of total gain: distance 68.9%, equipment 20.7%, weight 4.7%, the four coordinates 5.3% together, day of week 0.5%.

Two things are worth drawing out of that. Weight earns 4.7% despite a 0.03 correlation with the rate, because a tree model finds the effect *inside* a distance and equipment group, where a single correlation cannot. And the coordinates' 5.3% is an average over all loads. On a city the model has never seen, dropping location costs 17% of MAE (Section 5), so a low average share does not mean a feature is unimportant.

5. Validating the design decisions

A single holdout number does not show whether a design choice is right. Three decisions were tested separately, each over multiple windows so that the answer does not rest on one lucky period.

Does the holdout cover the real validation set?

Not entirely, and this is the most important check in the report. Every city in September and October was already present in training, but the real validation set contains **8 cities the model has never seen, behind 1,447 loads, or 12.1% of the score**. The holdout is blind to exactly the case that decides an eighth of the result.

So the case was rebuilt on purpose: 8 random cities hidden from training, 5 rounds, scoring only the holdout loads that use them.

Average MAE over 5 rounds	
City seen in training (reference)	\$97.21
New city, with coordinates	\$100.00
New city, no location at all	\$113.54

A new city costs \$2.79 (2.9%) with coordinates and \$16.33 (17%) without them, so coordinates recover 83% of the gap. This is why the model carries no city or lane identifiers. A model that memorizes city names has nothing to say about a city it has never seen, while coordinates place a new city next to the ones it already knows.

What should the model predict?

Same features, same model, three ways of writing the target, across three two-month windows:

Window	Rate per mile	Log rate	Dollars
May-June	193.25	195.07	193.97
July-August	103.85	105.16	105.48
September-October	97.97	99.72	99.80

Rate per mile wins all three, but only by 1-2%. A gradient-boosted model finds distance on its own, so the target shape matters far less here than it would for a linear model. It stays because it wins everywhere, because it is the unit the industry quotes in, and because it keeps the December chart readable as \$/mile.

Should the model extrapolate a trend?

A time trend lets the model carry the recent price level forward. Each window is predicted by a model trained only on what precedes it:

Window	Day of week only	Day of week + trend
May-June	193.25	112.35
July-August	103.85	202.66
September-October	97.97	107.93

The trend helps when prices keep moving the same way and hurts badly when they turn. It loses two of three windows, and its worst case doubles the error. Nobody knows which way November and December go, so the **decision is no trend**. Day of week is the only date feature, which means the model predicts the average level of the training period plus the weekly pattern.

6. The model

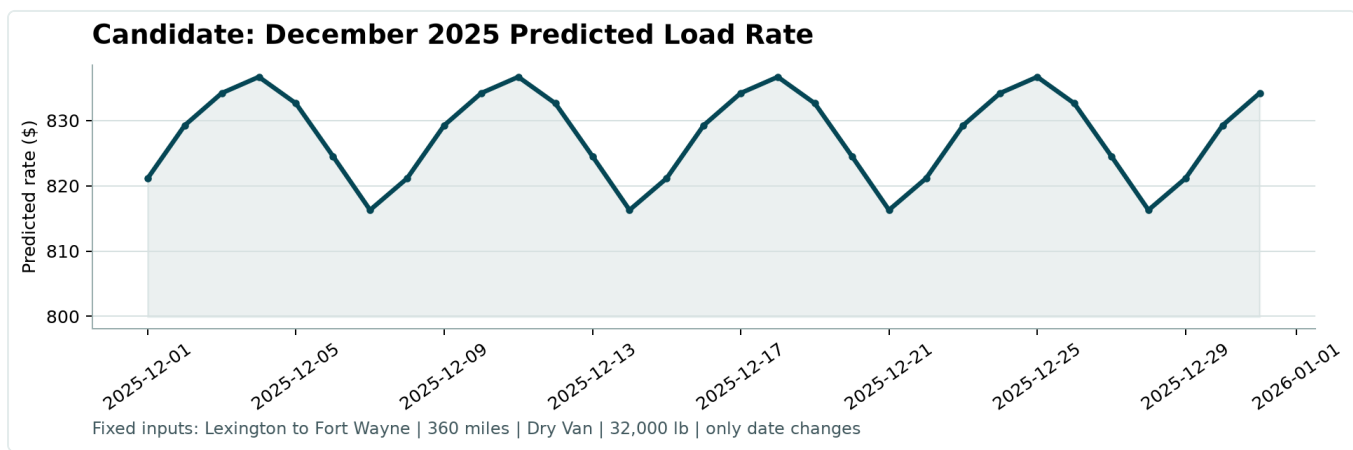
LightGBM regressor, 400 trees, learning rate 0.05, 31 leaves, minimum 50 samples per leaf.

The choice follows what the data is: tabular, mixed numeric and categorical, with interactions that matter (the same distance prices differently under Reefer than Dry Van) and missing values that should not force imputation. A linear model would need the distance relationship hand-built, and a neural network has nothing to offer 48,000 rows of tabular data that boosting does not.

The model fits rate per mile and multiplies back by distance. Features: distance, equipment, weight, the four coordinates, day of week. Every one of them is available in the December chart file or rebuildable for it, which was a hard constraint rather than a preference.

7. December prediction chart

Produced by the provided `score.py` from `data/december_chart_inputs.csv`. Fixed inputs: Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb. Only the date changes.



The chart shows a clean weekly cycle between \$816 and \$837, with no trend. That flatness is deliberate. Section 5 shows a trend losing two of three windows and doubling the error when the market turns, so the model carries the recent price level forward and adds the weekly pattern (midweek slightly more expensive, weekend lowest) instead of extrapolating a direction that nothing in the data justifies.

8. Reproducing

```
uv sync
uv run python -m src.evaluate # every table in Sections 4 and 5
uv run python -m src.predict  # writes both prediction files

python -m pip install -r requirements.txt
python score.py --predictions validation_predictions.csv \
                --december-predictions data/december_chart_inputs.csv
```

Full reasoning for every decision is in `DECISION_LOG.md`, and the exploration they came out of is in `notebooks/eda.ipynb`.